

AWS Infrastructure & App Deployment Bootcamp

Corporate Technical Workshop

By: Vicky Patil

Table of Contents

1. What is Cloud Computing?	6
.	6
2. Workshop Agenda	7
.	7
3. EC2 — Creating Your Ubuntu Server	8
.	8
4. SSH Connection via MobaXterm	10
.	10
5. Linux Fundamentals	10
.	10
6. System Update — Before Installing Anything	13
.	13
7. Installing the Development Stack	13
.	13
8. AWS CLI — Managing AWS from the Terminal	16
.	16
9. Pulling Code and Running a Python App	16
.	17
10. Systemd Services — Running Apps Permanently	19

..... 19

11. Nginx — Reverse Proxy Setup 22

..... 22

12. Complete Production Deployment Flow 24

..... 24

13. Quick Reference Card 25

..... 25

Prerequisites

Before attending this workshop, ensure you are comfortable with the following topics. You do not need to be an expert — a basic working knowledge of each is sufficient.

Git & GitHub

Know how to clone a repository, commit changes, and push to GitHub. Understand what a branch is and how to switch between them.

Python

Comfortable writing and running basic Python scripts. Understand functions, imports, and running a script from the terminal.

Python Virtual Environment (venv)

Know what a virtual environment is and why it is used. Should have created one at least once using `python3 -m venv`.

Nginx

Awareness of what a web server is. No configuration experience required — we will cover setup from scratch.

Linux / Ubuntu OS

Basic familiarity with the terminal — navigating folders, running commands. No advanced Linux knowledge needed.

Note: A 30-minute read or study of any unfamiliar topic before the workshop will get you familiar enough to follow along with confidence.

1. What is Cloud Computing?

Before cloud computing, every company that wanted to run software had to buy physical servers, install them in a data centre, hire staff to maintain them, and plan for capacity years in advance. This was expensive, slow, and hard to scale.

The Old Way (On-Premises)

- Companies purchased physical servers — large upfront capital expense
- Servers took weeks or months to procure and set up
- If traffic doubled overnight, you could not instantly add capacity
- Maintenance, hardware failures, and upgrades were your team's responsibility
- Servers sat idle during low-traffic periods — wasted money

The Cloud Way

Cloud providers like AWS, Azure, and Google Cloud own massive global data centres. You rent compute, storage, and services from them on demand — paying only for what you use, scaling up or down in minutes.

- No hardware to buy — rent virtual servers in minutes
- Pay per hour or per second — stop the server, stop the bill
- Scale instantly — add more servers with a few clicks
- Global reach — deploy in any region worldwide
- Managed infrastructure — AWS handles the physical layer

AWS — Amazon Web Services

AWS is the largest cloud provider in the world. It offers over 200 services. For most applications you will regularly use a small subset of them:

Service	What It Does	When You Use It
EC2	Virtual servers (Linux/Windows)	Host apps, APIs, scripts
S3	Object storage — store any file	Store datasets, model files, uploads
RDS	Managed relational databases	PostgreSQL, MySQL — no server management
Lambda	Serverless functions	Run code on triggers without a server
IAM	Identity & Access Management	Control who can access what
VPC	Virtual private network	Network isolation for your resources
Route 53	DNS management	Point domains to your EC2/S3
CloudWatch	Logs, metrics, alerts	Monitor your app and set alarms

Today's focus: EC2 — the service that lets you rent a Linux server and deploy your application on it. Everything else in this workshop runs on or connects to EC2.

2. Workshop Agenda

This workshop is hands-on. Every topic includes a live demonstration on a real AWS server. By the end, you will have a Python application running permanently on a cloud server, accessible via a clean URL — exactly as it runs in production.

#	Topic	What You Will Learn
A	Cloud Fundamentals	What cloud is, AWS overview, key services
B	EC2 Instance Setup	Create Ubuntu server, configure ports, assign Elastic IP
C	SSH Connection	Connect via MobaXterm using PEM key
D	Linux Fundamentals	File system, essential commands, permissions
E	Development Stack	Python, venv, Git, Docker, Nginx installation
F	Run Python App	Pull code, install dependencies, start app
G	Systemd Services	Run app permanently, auto-restart, auto-start on reboot
H	Nginx Reverse Proxy	Expose app on port 80, clean URL, no port number
I	Private GitHub Repos	Pull private repos using Personal Access Token

3. EC2 — Creating Your Ubuntu Server

Amazon EC2 (Elastic Compute Cloud) lets you rent a virtual Linux server in the cloud. You get full root access, can install anything, and control it completely over SSH. Think of it as your own Linux machine — just hosted in an AWS data centre.

Key Concepts — Know These Before You Start

Term	What It Means
AMI (Amazon Machine Image)	The OS template — like an installer disc. We use Ubuntu Server LTS.
Instance Type	The hardware spec. t2.micro = 1 vCPU, 1GB RAM. t3.medium = 2 vCPU, 4GB RAM.
Key Pair (.pem file)	Your SSH password. Generated once. If lost, you lose access. Store safely.
Security Group	Virtual firewall. Controls which ports are open to the internet.
Elastic IP	A permanent static IP. Without it, your server gets a new IP every reboot.
EBS (Elastic Block Store)	The hard disk. Data survives stop/restart. Lost only on termination.
Region	Geographic location. ap-south-1 = Mumbai. Choose closest to your users.

Creating an EC2 Instance — Step by Step

- **Step 1** — Log in to AWS Console → search EC2 → open EC2 Dashboard
- **Step 2** — Click the orange Launch Instance button
- **Step 3** — Set a name, choose Ubuntu Server (latest LTS), select t2.micro (free tier)
- **Step 4** — Create a new Key Pair → RSA type → .pem format → download and store safely
- **Step 5** — Configure Security Group — add inbound rules for ports 22, 80, 443
- **Step 6** — Leave storage at 8GB default (or increase for larger apps)
- **Step 7** — Click Launch Instance → wait for status: Running
- **Step 8** — Copy the Public IPv4 address from the instance details

Security Group — Required Port Rules

Port	Protocol	Purpose	Source
22	TCP	SSH — to connect via MobaXterm	My IP (recommended)
80	TCP	HTTP — for Nginx web traffic	0.0.0.0/0 (Anywhere)
443	TCP	HTTPS — for SSL/TLS traffic	0.0.0.0/0 (Anywhere)

Note: Restrict port 22 to 'My IP' only. Opening SSH to 0.0.0.0/0 exposes your server to brute-force attempts from the entire internet.

Elastic IP — Assign a Static IP

By default, EC2 gives your server a new public IP every time it restarts. This breaks saved MobaXterm sessions and DNS records. Assign an Elastic IP to keep it permanent.

- Go to EC2 → Elastic IPs → Allocate Elastic IP address → confirm

- Select the new IP → Actions → Associate Elastic IP → select your instance → save

Note: Elastic IP is free while associated with a running instance. Release it when you no longer need it — otherwise AWS charges for unassociated reserved IPs.

Instance States

State	Meaning	Charges?
Running	Active, accessible	Yes — compute charges apply
Stopped	Shut down, data preserved on EBS	No compute charge, storage still billed
Terminated	Permanently deleted, data gone	Nothing — instance no longer exists

Note: Always Stop — never Terminate — unless you are absolutely sure you no longer need the instance.

4. SSH Connection via MobaXterm

SSH (Secure Shell) is a protocol that lets you securely connect to and control a remote Linux server from your local machine. Everything you type locally is executed on the remote server.

What is a PEM File?

When you created the EC2 instance, AWS generated a Key Pair — a pair of cryptographic keys. AWS keeps the public key on the server. You downloaded the private key as a .pem file. This file is your authentication credential — it replaces a password.

- Downloaded exactly once during EC2 setup — never available again from AWS
- Store it in a dedicated folder: C:\Users\your-name\aws-keys\
- Never commit it to Git or share it over email or chat
- If you lose it, you lose SSH access — you must create a new key pair

What is MobaXterm?

MobaXterm is a Windows application that provides an SSH client, a built-in terminal, and a file browser for drag-and-drop file transfer to/from the server. It supports .pem files natively — no conversion needed.

Connecting via MobaXterm — Step by Step

- **Step 1** — Open MobaXterm → click Session (top-left corner)
- **Step 2** — Select SSH as the session type
- **Step 3** — Remote Host: enter the EC2 public IP | Specify Username: ubuntu | Port: 22
- **Step 4** — Click Advanced SSH settings tab → check Use private key → browse to your .pem file
- **Step 5** — Click OK → accept the fingerprint on first connection
- **Step 6** — Successful connection shows: ubuntu@ip-xxx-xxx-xxx:~\$
- **Step 7** — Session is saved in the left sidebar — double-click to reconnect anytime

Note: The default username for Ubuntu EC2 instances is ubuntu. Amazon Linux uses ec2-user. Debian uses admin.

5. Linux Fundamentals

Ubuntu is a Linux operating system. If you are used to Windows, Linux will feel different initially but follows a very logical structure. Everything is a file, and everything is organised under a single root directory.

Linux File System vs Windows

In Windows, drives are labelled C:\, D:\, etc. In Linux, everything starts from a single root /. Additional drives are mounted as subdirectories under /.

Windows	Linux Equivalent	Purpose
C:\	/	Root — the top of the entire file system

Windows	Linux Equivalent	Purpose
D:\ (second drive)	/mnt/d	Additional drives mounted here
C:\Program Files	/usr	Installed software and binaries
C:\Users	/home	User home directories
C:\Users\username	/home/ubuntu	Your personal home folder
C:\Windows\Temp	/tmp	Temporary files — cleared on reboot
C:\Windows\System32	/bin, /sbin	System commands
C:\ProgramData\configs	/etc	Configuration files for all services
C:\Windows\Logs	/var/log	Application and system log files

Important Directories to Know

Directory	What Lives Here
/	Root of the entire file system
/home/ubuntu	Your working directory — where you put your app code
/etc	Config files: Nginx configs, systemd service files live here
/var/log	Log files for Nginx, your app, system events
/usr	Installed packages and their binaries
/bin	Core system commands (ls, cp, mv, etc.)
/tmp	Temporary storage — cleared on reboot, never store important files here
/etc/systemd/system/	Where you create service files for your app
/etc/nginx/	Nginx configuration files

Essential Commands — Navigation

Command	What It Does	Example
pwd	Print working directory — shows where you are	pwd
ls	List files and folders	ls
ls -la	List all files including hidden, with details	ls -la
cd folder	Change into a directory	cd /home/ubuntu
cd ..	Go up one directory level	cd ..
cd ~	Go to your home directory	cd ~
mkdir name	Create a new directory	mkdir myapp
rm file	Delete a file	rm old.txt
rm -rf folder	Delete a folder and all its contents	rm -rf logs/
cp src dest	Copy a file	cp app.py backup.py
mv src dest	Move or rename a file	mv app.py main.py

Essential Commands — File Reading

Command	What It Does
cat file.txt	Print entire file contents to screen
nano file.txt	Open file in simple text editor — beginner friendly
vim file.txt	Open file in vim editor — press i to insert, :wq to save and exit
less file.txt	View file one page at a time — press q to quit
tail -f file.txt	Watch a file live — new lines appear as they are written (great for logs)
tail -n 50 file.txt	Show last 50 lines of a file

Essential Commands — Permissions

Linux has a permission system for every file and directory. Three categories: owner, group, others. Three permissions: read (r), write (w), execute (x).

Command	What It Does	Example
chmod 755 file	Set permissions (owner: rwx, others: rx)	chmod 755 app.py
chmod 600 file	Owner read/write only — used for .pem keys	chmod 600 mykey.pem
chown user file	Change file owner	chown ubuntu app.py
sudo command	Run command as root (administrator)	sudo apt install nginx

Essential Commands — Process Management

Command	What It Does
ps aux	List all running processes on the system
ps aux grep python	Filter process list — find your Python app
ps aux grep myapp	Check if your specific app process is running
kill PID	Stop a process by its process ID
kill -9 PID	Force-kill a process that won't stop normally
top	Live process monitor — CPU and memory usage (press q to quit)
htop	Better version of top — coloured, easier to read

Essential Commands — System Information

Command	What It Does
df -h	Show disk usage in human-readable format
free -h	Show RAM usage — total, used, free
uname -a	Show Linux kernel and OS version
whoami	Show current logged-in user
uptime	Show how long the server has been running

6. System Update — Before Installing Anything

Before installing any software on a fresh Ubuntu server, always update the package list and upgrade existing packages. This ensures you get the latest versions and security patches, and prevents version conflicts during installation.

The Package Manager — apt

Ubuntu uses apt (Advanced Package Tool) as its package manager. It is the equivalent of an app store for the command line. It downloads and installs software from official Ubuntu repositories.

Update Commands

```
# Step 1 – Update the package list
```

```
# This fetches the latest catalogue of available packages from Ubuntu's servers
```

```
# It does NOT install or change anything yet
```

```
sudo apt update
```

```
# Step 2 – Upgrade all installed packages to their latest versions
```

```
# The -y flag automatically confirms the upgrade without prompting
```

```
sudo apt upgrade -y
```

Command	What It Does
sudo apt update	Refreshes the package list from Ubuntu repositories — run this first always
sudo apt upgrade -y	Installs the latest version of all currently installed packages
sudo apt install X	Install a new package (e.g. sudo apt install python3)
sudo apt remove X	Uninstall a package
apt list --installed	List all installed packages

Note: Always run `sudo apt update` before any `sudo apt install` command. Skipping this can result in installing an outdated version or hitting dependency errors.

7. Installing the Development Stack

After updating the system, install the software stack your application needs. The commands below install each tool, explain what it is, and show how to verify the installation.

7.1 Python 3

Python is the runtime for your application. Ubuntu Server ships with Python 3 pre-installed but the package manager version should be explicitly installed to ensure you have the tools you need.

```
sudo apt install python3 -y
```

```
python3 --version          # Verify – should show Python 3.x.x
```

Note: Always use python3 on Ubuntu. The command python (without 3) may point to Python 2 or not exist.

7.2 pip — Python Package Installer

pip is the tool that installs Python packages (libraries). Your app's requirements.txt is installed using pip.

```
sudo apt install python3-pip -y
```

```
pip3 --version            # Verify
```

7.3 Python Virtual Environment (venv)

A virtual environment is an isolated Python installation for a single project. It prevents version conflicts between projects — each project has its own set of packages, independent of the system Python and other projects.

Why virtual environments are essential in production:

- Project A needs Django 3.2, Project B needs Django 4.1 — venv lets both coexist
- System Python packages can conflict with your app's dependencies
- When you create a service file, it points directly to the venv Python — isolated and predictable
- Deleting a venv is clean — no system-wide side effects

```
sudo apt install python3-venv -y
```

```
# Navigate to your project folder
```

```
cd /home/ubuntu/myapp
```

```
# Create the virtual environment (creates a 'venv' folder)
```

```
python3 -m venv venv
```

```
# Activate the virtual environment
```

```
source venv/bin/activate
```

```
# Your prompt changes to show (venv) – you are now inside the venv
```

```
# (venv) ubuntu@ip-xxx:~/myapp$
```

```
# Install your app's dependencies
```

```
pip install -r requirements.txt
```

```
# Deactivate when done (returns to system Python)
```

```
deactivate
```

Command	What It Does
python3 -m venv venv	Create virtual environment in a folder named 'venv'
source venv/bin/activate	Activate — all pip installs go into this env
deactivate	Exit the virtual environment
pip install -r requirements.txt	Install all packages listed in requirements file
pip list	Show all installed packages in the active env
which python	Confirm Python is running from venv, not system

7.4 Git

Git is the version control system. You use it to pull your application code from GitHub onto the server.

```
sudo apt install git -y
git --version          # Verify
```

7.5 Docker

Docker packages your application and all its dependencies into a container — a portable, isolated unit that runs identically on any machine. Today we install it for completeness; Docker-based deployments are covered in a separate session.

```
sudo apt install docker.io -y

# Start Docker service
sudo systemctl start docker

# Enable Docker to start on every reboot
sudo systemctl enable docker

# Verify
docker --version
```

Note: To run Docker commands without sudo, add your user to the docker group: `sudo usermod -aG docker ubuntu` — then log out and back in.

7.6 Nginx

Nginx is a web server and reverse proxy. It sits in front of your app and handles all incoming internet traffic, forwarding it to your app's internal port. This is covered in detail in Section 9.

```
sudo apt install nginx -y
sudo systemctl start nginx
sudo systemctl enable nginx
nginx -v          # Verify
```

Installation Verification — Quick Reference

Software	Install Command	Verify Command
Python 3	<code>sudo apt install python3 -y</code>	<code>python3 --version</code>
pip	<code>sudo apt install python3-pip -y</code>	<code>pip3 --version</code>
venv	<code>sudo apt install python3-venv -y</code>	<code>python3 -m venv --help</code>
Git	<code>sudo apt install git -y</code>	<code>git --version</code>
Docker	<code>sudo apt install docker.io -y</code>	<code>docker --version</code>
Nginx	<code>sudo apt install nginx -y</code>	<code>nginx -v</code>

8. AWS CLI — Managing AWS from the Terminal

The AWS CLI (Command Line Interface) lets you control AWS services directly from the terminal without opening the web console. You can upload files to S3, manage EC2 instances, retrieve secrets, and more — all from your server.

Why Use AWS CLI on EC2?

- Upload app outputs directly to S3 from a script
- Pull configuration or model files from S3 at startup
- Automate tasks — no need to log into the console manually
- Works with IAM roles — no credentials needed when configured correctly

Installation

```
sudo apt install awscli -y
aws --version          # Verify
```

Configuration

If your EC2 instance has an IAM Role attached with the required permissions, the CLI uses those credentials automatically — no manual configuration needed. This is the recommended approach.

If configuring manually (not recommended for production):

```
aws configure
# You will be prompted for:
# AWS Access Key ID:      [your access key]
# AWS Secret Access Key:  [your secret key]
# Default region name:    ap-south-1
# Default output format:  json
```

Common AWS CLI Commands

Command	What It Does
aws s3 ls	List all S3 buckets in your account
aws s3 ls s3://bucket-name/	List contents of a specific bucket
aws s3 cp file.txt s3://bucket/	Upload a file to S3
aws s3 cp s3://bucket/file.txt ./	Download a file from S3
aws s3 sync ./folder s3://bucket/folder/	Sync local folder to S3
aws ec2 describe-instances	List all EC2 instances

9. Pulling Code and Running a Python App

9.1 Clone a Public Repository

```
cd /home/ubuntu
git clone https://github.com/your-org/your-repo.git
cd your-repo
ls
```

9.2 Clone a Private Repository

Private repositories require authentication. GitHub no longer accepts passwords — you use a Personal Access Token (PAT) embedded in the URL.

Generating a PAT on GitHub:

- Step 1: Go to GitHub → Profile icon (top right) → Settings
- Step 2: Scroll to the bottom → Developer settings
- Step 3: Personal access tokens → Tokens (classic) → Generate new token (classic)
- Step 4: Select scopes: at minimum tick repo
- Step 5: Click Generate token — copy it immediately (shown only once)
- Step 6: If the repo belongs to an organisation, click Authorize SSO next to the token

```
# Clone main branch
git clone https://:@github.com//.git

# Clone a specific branch
git clone -b https://:@github.com//.git

# Example
git clone https://yourname:ghp_abc123token@github.com/YourOrg/YourRepo.git
```

Note: Never commit the token into any file. Clear your shell history after running the clone command: history -c

9.3 Set Up Virtual Environment and Install Dependencies

```
cd /home/ubuntu/your-repo

# Create virtual environment
python3 -m venv venv

# Activate
source venv/bin/activate

# Install dependencies
pip install -r requirements.txt
```

9.4 Run the App

```
# Run directly (for testing only – app dies when terminal closes)
python app.py

# Or for FastAPI/uvicorn
uvicorn main:app --host 0.0.0.0 --port 8085

# Or for Streamlit
streamlit run app.py --server.port 8085 --server.address 0.0.0.0
```

At this point, your app is reachable at **http://<server-ip>:8085**. But if you close the terminal, the app stops. The next section covers how to run it permanently.

9.5 localhost vs 0.0.0.0 vs Public IP

Address	Meaning	When to Use
localhost / 127.0.0.1	The server itself — not reachable from outside	Testing from within the server
0.0.0.0	Bind to all network interfaces — accept from anywhere	Required when hosting for external access
EC2 Public IP	The address users type in their browser	Share this with your users

Note: When you run your app with `--host 0.0.0.0`, you are telling the app to accept connections from any IP — not just localhost. This is required for external users to reach your app.

10. Systemd Services — Running Apps Permanently

When you run a Python app directly in the terminal, it stops the moment you close the terminal or your SSH session disconnects. In production, your app must run continuously — surviving reboots, crashes, and disconnects.

Linux's systemd is the solution. Systemd is the process manager built into Ubuntu. When you register your app with systemd, Linux manages it exactly like any system service: Nginx, Docker, SSH — they all run as systemd services. Now your app does too.

What a Service Gives You

Feature	What It Means
Runs in background	App continues running after you close the terminal
Auto-restarts on crash	If your app dies due to an error, Linux restarts it automatically
Auto-starts on reboot	Server restarts after maintenance — app comes back up without you
Centrally managed	Start / stop / restart using one consistent command
Built-in logging	All app output captured — view with journalctl

Creating the Service File

Service files live in `/etc/systemd/system/`. Create one file per app.

```
sudo nano /etc/systemd/system/myapp.service
```

Paste the following and update the paths:

```
[Unit]
Description=My Python Web App
After=network.target

[Service]
User=ubuntu
WorkingDirectory=/home/ubuntu/myapp
ExecStart=/home/ubuntu/myapp/venv/bin/python app.py
Restart=always

[Install]
WantedBy=multi-user.target
```

Save and exit: `Ctrl + X → Y → Enter`

Service File Fields Explained

Field	What It Does
Description	A human-readable label — appears in <code>systemctl status</code> output
After=network.target	Wait for network to be ready before starting (important for web apps)

Field	What It Does
User=ubuntu	The Linux user account that runs the app
WorkingDirectory	The project root folder — relative paths in your app resolve from here
ExecStart	The exact command to start your app — must use the full venv Python path
Restart=always	Automatically restart the app if it exits for any reason
WantedBy=multi-user.target	Enables auto-start on system boot

ExecStart Examples

App Type	ExecStart Line
Python script	/home/ubuntu/myapp/venv/bin/python app.py
FastAPI (uvicorn)	/home/ubuntu/myapp/venv/bin/uvicorn main:app --host 0.0.0.0 --port 8085
Streamlit	/home/ubuntu/myapp/venv/bin/streamlit run app.py --server.port 8085 --server.address 0.0.0.0

Starting and Managing the Service

```
# Tell systemd to re-read all service files (run after every service file change)
```

```
sudo systemctl daemon-reload
```

```
# Start the app
```

```
sudo systemctl start myapp
```

```
# Check if it is running
```

```
sudo systemctl status myapp
```

```
# Enable auto-start on reboot (run this once per service)
```

```
sudo systemctl enable myapp
```

Daily Service Management Commands

Action	Command
Start app	sudo systemctl start myapp
Stop app	sudo systemctl stop myapp
Restart app	sudo systemctl restart myapp
Check status	sudo systemctl status myapp
Enable on reboot	sudo systemctl enable myapp
Disable on reboot	sudo systemctl disable myapp
Reload daemon	sudo systemctl daemon-reload
View live logs	journalctl -u myapp -f
View last 50 log lines	journalctl -u myapp -n 50

Checking Running Processes

To check if your app process is actually running at the OS level (not just systemd reporting it):

```
# List all processes and filter for your app name
```

```
ps aux | grep python

# Or filter by script name
ps aux | grep app.py

# Output columns: USER  PID  CPU%  MEM%  ...  COMMAND
# PID is the process ID – use it to kill the process if needed
```

Reading Logs with journalctl

Every service managed by systemd writes its output to the systemd journal. journalctl is the tool to read these logs.

```
# Follow live log output (like tail -f)
journalctl -u myapp -f

# Show last 50 lines
journalctl -u myapp -n 50

# Show logs since the last boot
journalctl -u myapp -b

# Show logs with timestamps
journalctl -u myapp --since "2024-01-01" --until "2024-01-02"
```

11. Nginx — Reverse Proxy Setup

Nginx is a web server and reverse proxy. Once your app is running as a systemd service on an internal port (e.g. 8085), Nginx sits in front of it and handles all public traffic. Users visit a clean URL — Nginx invisibly routes their request to your app.

The Problem Without Nginx

Without Nginx, users must type the port in the URL:

```
http://54.123.45.67:8085 ← ugly, exposes port, not professional
```

Problem	Why It Matters
Port in URL	Unprofessional, users forget it, hard to share
Can't use a custom domain	DNS cannot point a domain to a specific port
No HTTPS	Browser shows 'Not Secure' warning
All app ports must be open	Security risk — more exposed ports means larger attack surface
Multiple apps are complex	Hard to host 2 apps on same server without Nginx routing

How Nginx Reverse Proxy Works

Internet Traffic

↓

Port 80 (HTTP) → Nginx receives request

↓

Nginx reads config file → matches server_name and location

↓

proxy_pass → forwards to app on localhost:8085

↓

App responds → Nginx sends response back to user

User sees: http://your-ip (no port needed)

Nginx Config File — Setup

```
# Create the config file for your app
```

```
sudo nano /etc/nginx/sites-available/myapp
```

Paste this config (replace 8085 with your app's port):

```
server {  
    listen 80;  
    server_name _;          # _ means catch-all – matches any IP or domain  
  
    location / {  
        proxy_pass http://localhost:8085;  
        proxy_http_version 1.1;  
    }  
}
```

```

    proxy_set_header Upgrade $http_upgrade;
    proxy_set_header Connection "upgrade";
    proxy_set_header Host $host;
    proxy_set_header X-Real-IP $remote_addr;
}
}

```

Config Line Explanations

Line	What It Does
listen 80;	Accept all traffic on port 80 (standard HTTP)
server_name _;	Match all incoming hostnames — use your domain here once you have one
location / {	Apply these rules to all URL paths
proxy_pass http://localhost:8085;	Forward requests to your app's internal port
proxy_http_version 1.1;	Use HTTP/1.1 — required for WebSocket support
Upgrade \$http_upgrade;	Allow HTTP connections to upgrade to WebSocket
Connection "upgrade";	Keep WebSocket connections alive
Host \$host;	Pass original host header to app
X-Real-IP \$remote_addr;	Pass user's real IP to app (otherwise app sees only Nginx's IP)

Enable and Activate the Config

```

# Create symlink – this activates the config
sudo ln -s /etc/nginx/sites-available/myapp /etc/nginx/sites-enabled/

# Test config for syntax errors – ALWAYS do this before reloading
sudo nginx -t

# Expected output:
# nginx: configuration file /etc/nginx/nginx.conf syntax is ok
# nginx: configuration file /etc/nginx/nginx.conf test is successful

# Reload Nginx (applies new config with zero downtime)
sudo systemctl reload nginx

# Test in browser – no port needed
# http://your-elastic-ip

```

Nginx Daily Commands

Action	Command
Start	sudo systemctl start nginx
Stop	sudo systemctl stop nginx
Restart	sudo systemctl restart nginx
Reload (no downtime)	sudo systemctl reload nginx
Check status	sudo systemctl status nginx

Action	Command
Test config	sudo nginx -t
View error logs	sudo tail -f /var/log/nginx/error.log
View access logs	sudo tail -f /var/log/nginx/access.log

Troubleshooting Nginx

Symptom	Cause	Fix
502 Bad Gateway	Your app is not running	<code>sudo systemctl start myapp</code>
Connection refused	Nginx down or port 80 closed	Check <code>systemctl status nginx</code> and Security Group
Config syntax error	Typo in config file	<code>sudo nginx -t</code> shows the exact error line
App loads but no real-time	WebSocket headers missing	Verify Upgrade and Connection headers are present

12. Complete Production Deployment Flow

This section brings everything together into the sequence you will follow every time you deploy a Python app to a new EC2 server.

```
# ■■■■ 1. CONNECT ■■■■  
# SSH into your EC2 instance via MobaXterm  
  
# ■■■■ 2. UPDATE ■■■■  
sudo apt update && sudo apt upgrade -y  
  
# ■■■■ 3. INSTALL STACK ■■■■  
sudo apt install python3 python3-pip python3-venv git nginx -y  
  
# ■■■■ 4. PULL CODE ■■■■  
cd /home/ubuntu  
git clone https://username:TOKEN@github.com/org/repo.git  
cd repo  
  
# ■■■■ 5. SETUP VIRTUAL ENVIRONMENT ■■■■  
python3 -m venv venv  
source venv/bin/activate  
pip install -r requirements.txt  
deactivate  
  
# ■■■■ 6. CREATE SYSTEMD SERVICE ■■■■  
sudo nano /etc/systemd/system/myapp.service  
# (paste service file content, update paths, save)  
  
sudo systemctl daemon-reload  
sudo systemctl start myapp
```


Action	Command
Test config	<code>sudo nginx -t</code>
View error log	<code>sudo tail -f /var/log/nginx/error.log</code>

Python / venv

Action	Command
Create venv	<code>python3 -m venv venv</code>
Activate venv	<code>source venv/bin/activate</code>
Install deps	<code>pip install -r requirements.txt</code>
Deactivate	<code>deactivate</code>

File Structure Quick Map

Path	What It Is
<code>/home/ubuntu/</code>	Your working directory — put app code here
<code>/etc/systemd/system/myapp.service</code>	Your service file
<code>/etc/nginx/sites-available/myapp</code>	Your Nginx config file (draft)
<code>/etc/nginx/sites-enabled/myapp</code>	Symlink — active config
<code>/var/log/nginx/error.log</code>	Nginx error log
<code>/var/log/nginx/access.log</code>	Nginx access log

Homework — Private Repo Methods

Try pulling a private repository on your own using each method:

- Method 1 (shown today): Personal Access Token in the clone URL
- Method 2 (recommended for long-term): SSH Key authentication
- Method 3: GitHub Deploy Keys — repository-specific read-only access

Note: SSH key method is more secure for production — token in URL is visible in shell history. See the full guide in [05_git_clone_private_repo.md](#)